

本科（工科类）实验实践达标测试（A 级）自主选题申请表

姓 名	王舒贤	学 号	22009290060
所在学院	计算机科学与技术学院	联系电话	13402992020
项目名称	基于 stm32 的智能小车设计		
起止时间	2025 年 3 月至 2025 年 6 月		
项目来源 (见备注)	大三嵌入式综合实践设计		
项目情况及 个人完成 内容简介 (<300 字)	设计并实现了一款基于 STM32F103C8T6 微控制器的小车,具备蓝牙遥控、超声波避障与红外循迹等功能。项目采用模块化设计思想,将主控、电机驱动、感应器件、通信模块等合理分工,兼顾了功能实现与成本控制。软件方面使用 Keil5 平台开发,使用 C 语言编写程序逻辑,实现对硬件模块的控制和交互。该小车可通过手机蓝牙进行远程操控,也可在自动模式下实现对障碍物的检测避让以及对地面黑线的识别循迹,具备一定的智能性和扩展性。 个人完成代码编写, 小车程序设计及 pcb 版绘制 本人签名: 王舒贤 2025 年 9 月 11 日		
指导教师或企 业导师意见	签名: 单位: 年 月 日		
实验中心 审核意见	负责人签名: 年 月 日		



西安电子科技大学

计算机科学与技术学院

嵌入式应用综合设计技术报告

题目： 基于 STM32 的智能小车设计

姓名： 杨昕捷 学号 22009201392

姓名： 王舒贤 学号 22009290060

姓名： 刘魏征 学号 22009201004

指导老师： 吴自力

起止时间： 2025 年 3 月 至 2025 年 6 月

西安电子科技大学计算机工程系

2024 年 6 月制

摘要

本项目设计并实现了一款基于 STM32F103C8T6 微控制器的小车，具备蓝牙遥控、超声波避障与红外循迹等功能。项目采用模块化设计思想，将主控、电机驱动、感应器件、通信模块等合理分工，兼顾了功能实现与成本控制。软件方面使用 Keil5 平台开发，使用 C 语言编写程序逻辑，实现对硬件模块的控制和交互。该小车可通过手机蓝牙进行远程操控，也可在自动模式下实现对障碍物的检测避让以及对地面黑线的识别循迹，具备一定的智能性和扩展性。该项目不仅提高了对嵌入式系统开发流程的理解，也为后续从事机器人与智能车相关研发打下了良好基础。

第一章 绪论

1.1 设计目的

随着智能控制和嵌入式系统的快速发展，智能小车作为其综合应用的重要实践平台，越来越受到高校教学与科研的重视。通过本项目的设计与实现，能够加深对 STM32 嵌入式控制系统的理解，掌握基本外设模块的接口通信、信号采集与控制方法，并通过系统集成的方式，完成具有实用价值和一定智能行为的小车作品。

1.2 研究意义

本项目不仅有助于锻炼嵌入式开发能力、模块系统集成能力、软硬件调试能力，而且在智能交通、服务机器人等领域具有较高的研究借鉴价值。通过蓝牙遥控、超声波避障、红外循迹三种典型功能模块的协作，能够模拟现实中自动驾驶中的一部分控制逻辑，具有良好的扩展性与学习价值。

1.3 国内外研究现状

目前，智能小车的研究主要集中在高校与科研机构，国内已有许多高校在“智能车竞赛”背景下进行深入探索。国外在自动驾驶控制算法、视觉导航等方面已有较成熟研究，但在教学与入门实践层面，仍以模块化、嵌入式控制为主。相比于大型工业自动车，本项目设计的小车结构简洁、成本低廉、功能清晰，适合作为嵌入式开发初学者的训练平台。

第二章 系统需求分析

2.1 功能需求

本项目小车具备以下功能：

蓝牙遥控：可通过手机蓝牙终端发送控制指令，实现前进、后退、左转、右转、停止等基本操作。

超声波避障：通过超声波模块获取前方障碍物距离信息，在自动模式下进行避让。

红外循迹：采用 TCRT5000 模块检测地面黑线，实现对路线的自主跟随。

模式切换：可通过程序设置或蓝牙指令，在手动控制与自动运行之间切换。

2.2 性能需求

实时性：系统需实时响应用户遥控指令，并在自动模式下快速做出避障或转向判断。

稳定性：各个模块在长时间运行中应保持稳定工作，避免卡顿或误动作。

低功耗与长续航：通过优化电源管理，提高电池供电效率，延长小车运行时间。

成本控制：总成本应控制在学生可以承担的范围，同时保证功能的完整实现。

2.3 其他需求

易维护性：各模块采用标准接口连接，便于替换和调试。

可扩展性：预留接口或代码结构便于后续扩展，如加入摄像头、WiFi 模块等。

安全性：电源接线合理，防止短路与过载，程序运行中应有基本故障检测机制

第三章 系统软硬件设计与实现

3.1 方案论证与比较

在设计智能小车系统时，需首先在主控平台、电机驱动方式、传感器选择等方面进行充分论证。以下是本项目几种主要实现方案的对比分析：

方案	优点	缺点	本项目是否采用
主控 MCU: Arduino UNO	编程简单，社区资源丰富	性能不足，I/O 口较少	否
主控 MCU: STM32F103C8T6	性能强大，Cortex-M3 架构，资源丰富，低功耗	编程复杂，学习曲线较陡	✓ 是
电机驱动: L298N	市面常见，使用方便	占板面积大，效率低	否

方案	优点	缺点	本项目是否采用
电机驱动：DRV8833/TB6612	驱动能力强，体积小，效率高	低	
避障传感器：红外对射	成本低，响应快	接线稍复杂	✓ 是
避障传感器：超声波 HC-SR04	测距精度高，应用广泛	易受环境干扰，测距短	否
		安装需云台辅助避盲区	✓ 是

3.2 系统硬件设计

硬件实际连线图如下：

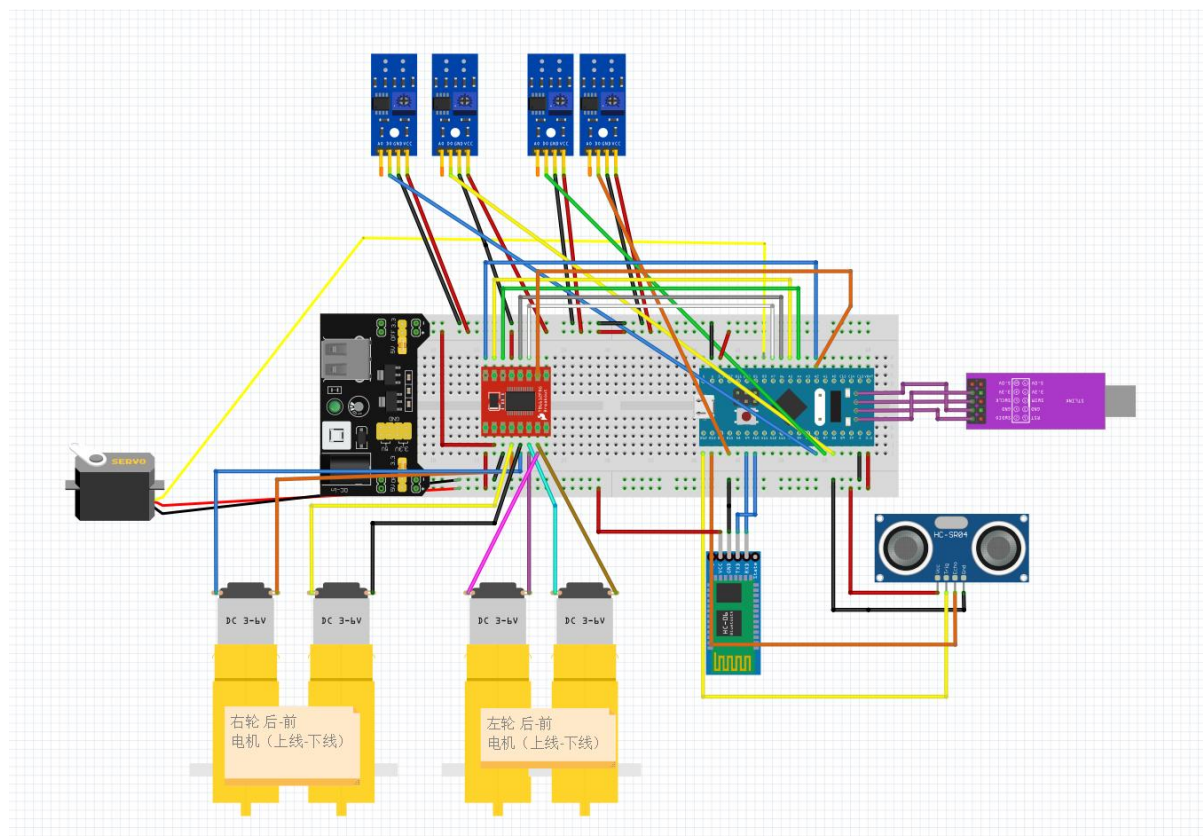


图 1 元件接线图

主要分为主控模块（STM32F103T8C6），电机驱动模块，蓝牙遥控模块，超声波避障模块，红外循迹模块

硬件框图如下：

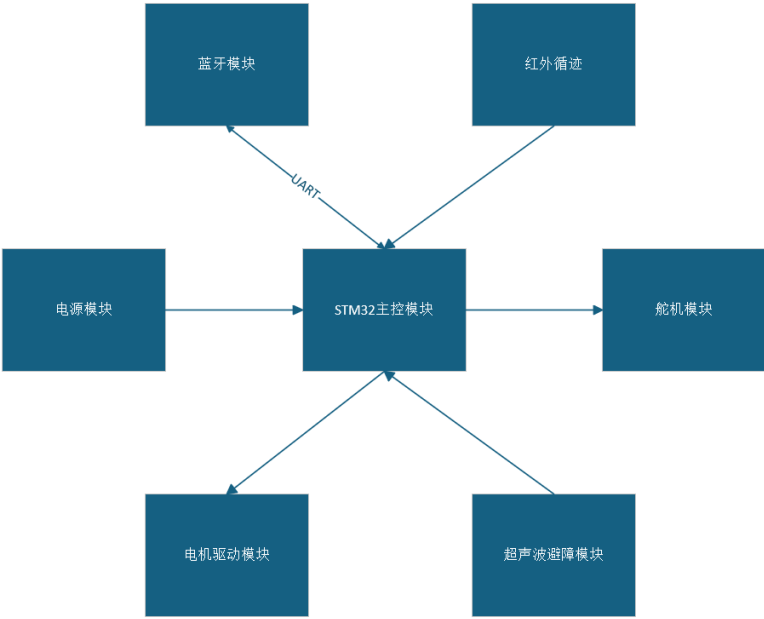


图 2 硬件框图

电路原理图如下：

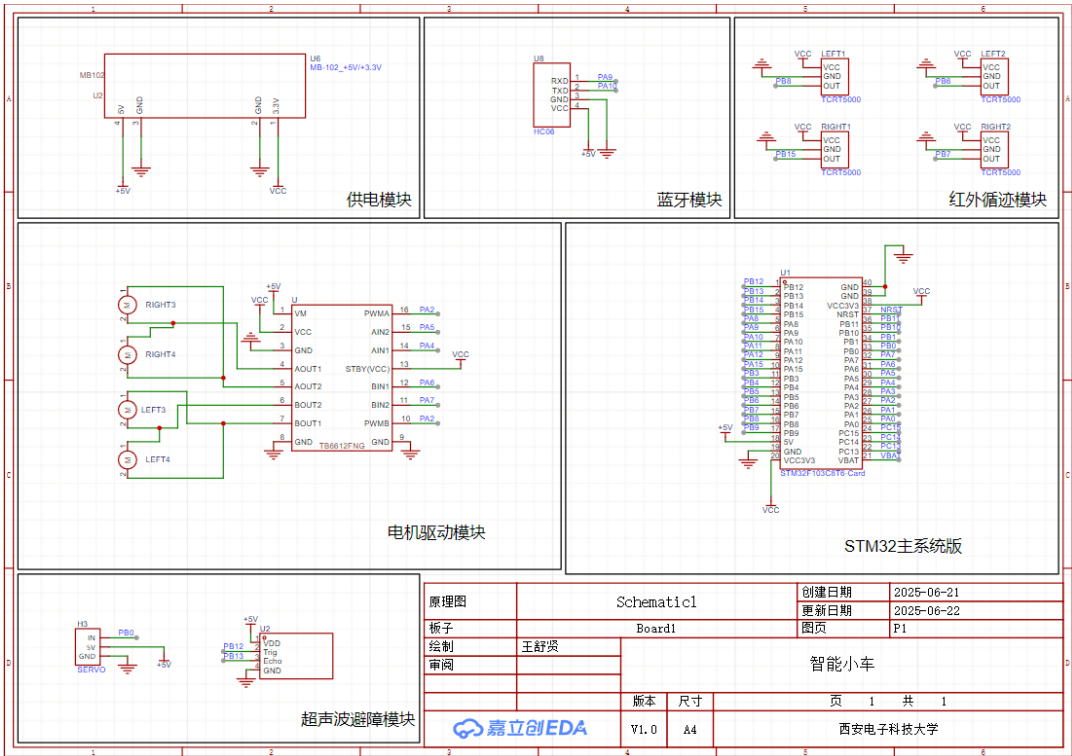


图 3 电路原理图

主控模块

STM32F103C8T6 是基于 ARM Cortex-M3 内核的 32 位单片机，主频 72MHz，具备丰富的外设接口（如 USART、GPIO、TIM、ADC 等），适合用于嵌入式控制场景。它作为整个小车系统的核心控制单元，负责读取传感器数据、处理控制逻辑、驱动电机与执行机构。

主要功能：

- 控制电机运行状态与方向
- 读取红外模块的循迹数据
- 读取超声波模块的距离信息
- 控制舵机转向
- 接收蓝牙指令并进行解析处理

为确保小车的各个模块能够稳定供电，使用了面包板电源模块或稳压模块进行电压转换和分配。

原理说明：

- 电池盒输出 6V 左右电压，供给面包板电源模块
- 面包板电源模块通过 LD0 稳压器转换为 3.3V/5V 供 STM32 与传感器模块使用
- 必要时增加电容滤波，避免电压波动影响系统稳定性

电机驱动模块是 DRV8833

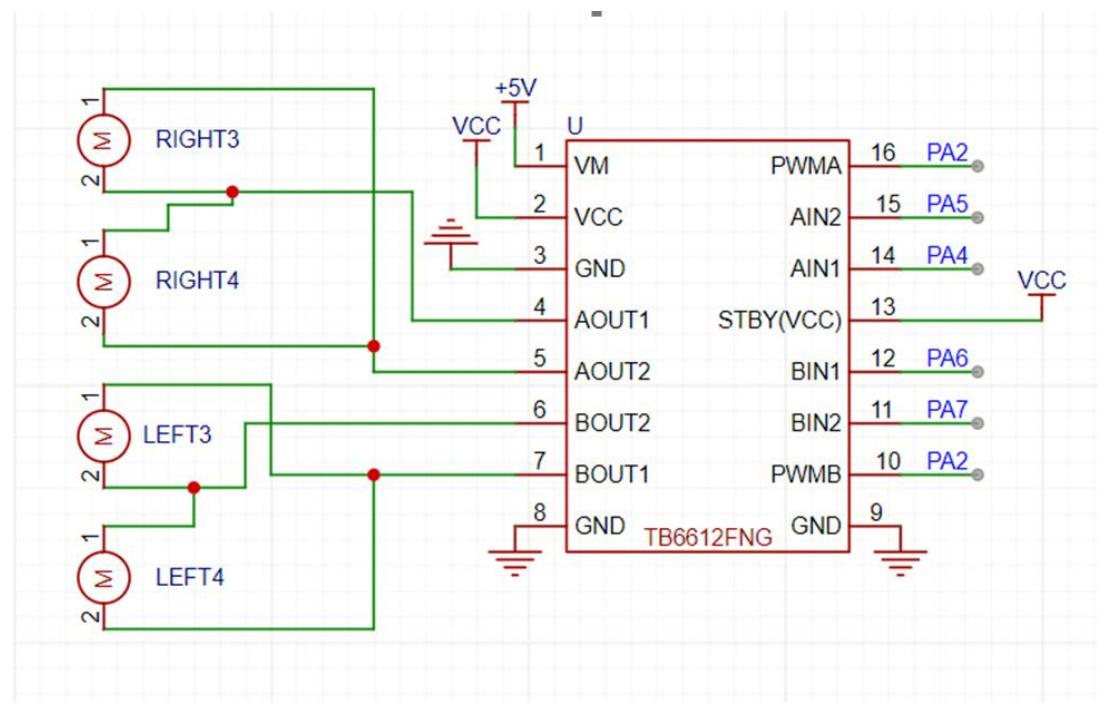


图 4 电机驱动模块

我们要使用定时器 (TIM) 2 的通道 (Channel) 2，使用 PWMA 控制左边两个轮子，PWMB 控制右边两个轮子

这里使用 pa4、pa5、pa6、pa7 做控制四个电机，根据 Aspet 的值控制左边与右边的

轮子

蓝牙模块

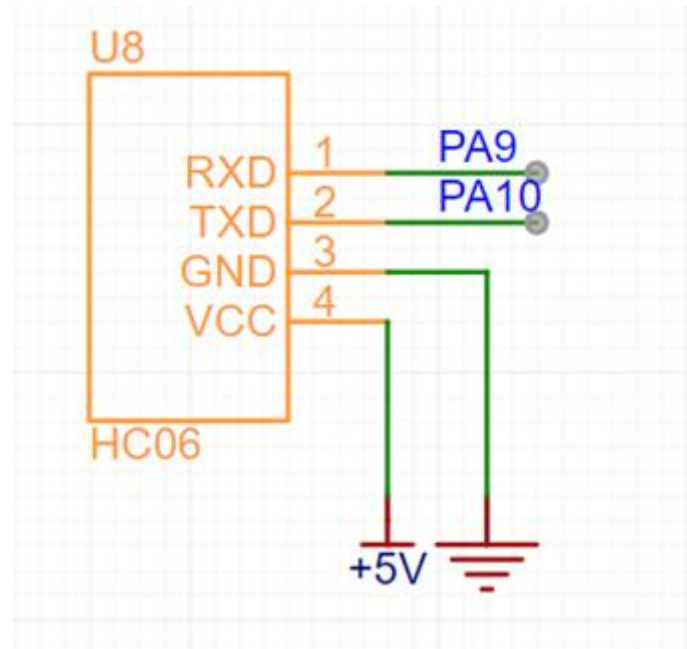


图 5 蓝牙模块

HC-06 为常见的蓝牙串口透传模块，实现小车与手机之间的无线通信。

原理说明：

- 模块通过 UART 串口与 STM32 通信（通常为 USART1）
- 手机端通过蓝牙调试助手发送 ASCII 字符控制指令（如‘F’前进、‘B’后退等）
- STM32 解析串口接收内容，根据命令控制电机动作

自动避障模块

舵机与超声波模块联合使用，实现检查前面是否有障碍物，如有，左拐或右拐，如果遇到死胡同（前左右都有障碍物）停下来

SG90 是一种常见的 180° 微型舵机，采用 PWM 控制信号来设定角度，用于驱动超声波模块云台实现方向扫描。

原理说明：

- STM32 输出周期为 20ms 的 PWM 信号（50Hz）
- 占空比决定舵机旋转角度（通常 1ms~2ms 对应 0°~180°）
- STM32 通过调整 PWM 占空比控制舵机旋转至左、中、右方向，配合超声波实现左右探测

超声波模块通过发射超声波脉冲并接收其回波来测量前方障碍物的距离。

原理说明：

- STM32 向模块的 Trig 脚发送一个 $10\mu\text{s}$ 的高电平脉冲
- 模块自动发送 8 个 40kHz 的超声波信号
- 接收脚 Echo 输出一个与回波时间成比例的高电平信号
- STM32 通过定时器测量高电平持续时间，计算距离：

距离 (cm) = 高电平时间 (μs) $\times 0.058$

距离 (cm) = 0.058 高电平时间 (μs)

- 超声波模块安装在云台上，由舵机带动左右旋转，以扩展探测视野



图 6 sg-90 舵机

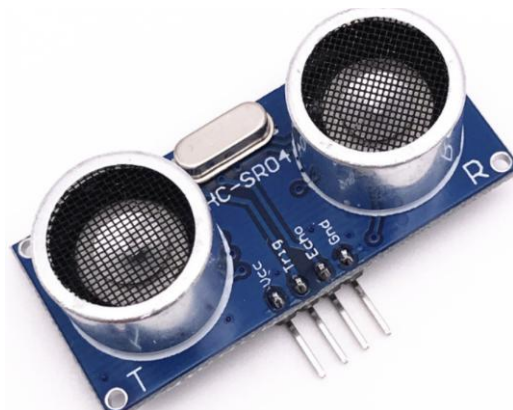


图 7 HC-SR04

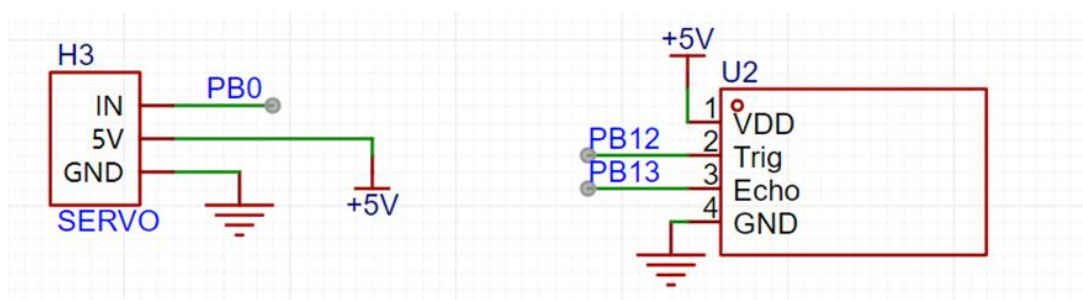


图 8 超声波避障模块

红外循迹模块（TCRT5000）

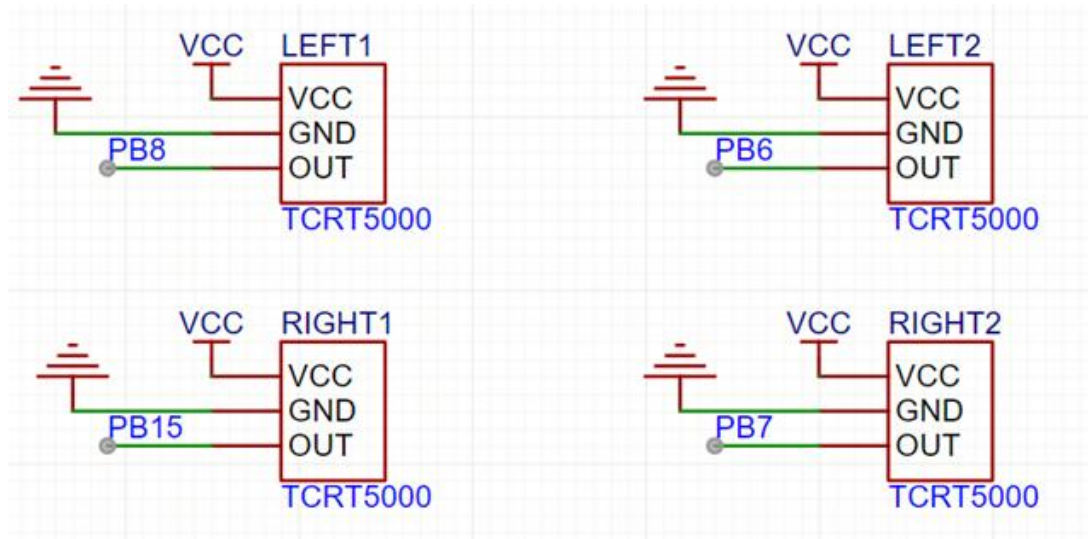


图9 红外循迹模块

红外循迹模块利用红外光的反射特性识别黑色线路，用于小车巡线导航。

原理说明：

- 红外发射管向地面发射红外光
- 黑线吸收红外光，白地反射红外光
- 接收管检测反射强度，并输出高/低电平信号
- STM32 读取各路模块的信号，通过逻辑判断小车是否偏离黑线，并进行相应调整

3.3 系统软件设计

电机驱动模块

我们要使用定时器(TIM) 2 的通道(Channel) 2，使用 PWMA 控制左边两个轮子，PWMB 控制右边两个轮子

这里使用 pa4、pa5、pa6、pa7 做控制四个电机，根据 Aspet 的值控制左边与右边的轮子

```

01. void Motor_SetSpeed(unsigned char Aspet,int8_t Speed)
02. {
03.     if(Aspet == 1)
04.     {
05.         if(Speed > 0)
06.         {
07.             GPIO_SetBits(GPIOA, GPIO_Pin_6);
08.             GPIO_ResetBits(GPIOA, GPIO_Pin_7);
09.             PWM_SetCompare(3,Speed);
10.         }else
11.         {
12.             GPIO_ResetBits(GPIOA, GPIO_Pin_6);
13.             GPIO_SetBits(GPIOA, GPIO_Pin_7);
14.             PWM_SetCompare(3,-Speed);
15.         }
16.     }else if(Aspet == 2)
17.     {
18.         if(Speed > 0)
19.         {
20.             GPIO_SetBits(GPIOA, GPIO_Pin_4);
21.             GPIO_ResetBits(GPIOA, GPIO_Pin_5);
22.             PWM_SetCompare(2,Speed);
23.         }else
24.         {
25.             GPIO_ResetBits(GPIOA, GPIO_Pin_4);
26.             GPIO_SetBits(GPIOA, GPIO_Pin_5);
27.             PWM_SetCompare(2,-Speed);
28.         }
29.     }
30. }

```

图 10 代码-电机驱动定义

根据 Motor_SetSpeed 的 Speed 参数正负值控制前进与后退、Aspet 控制左轮与右轮，新建一个车模块，封装前进、后退、左拐、右拐、后左拐、后右拐等功能。

```

01. void Car_Init(void){
02.     Motor_Init();
03. }
04. void Go_Ahead(void){
05.     Motor_SetSpeed(1,50);
06.     Motor_SetSpeed(2,50);
07. }
08. void Go_Back(void){
09.     Motor_SetSpeed(1,-50);
10.     Motor_SetSpeed(2,-50);
11. }
12. void Turn_Left(void){
13.     Motor_SetSpeed(1,0);
14.     Motor_SetSpeed(2,90);
15. }
16. void Turn_Right(void){
17.     Motor_SetSpeed(2,0);
18.     Motor_SetSpeed(1,90);
19. }
20. }
21. void Self_Left(void){
22.     Motor_SetSpeed(1,-90);
23.     Motor_SetSpeed(2,90);
24. }
25. void Self_Right(void){
26.     Motor_SetSpeed(1,90);
27.     Motor_SetSpeed(2,-90);
28. }
29. void Car_Stop(void){
30.     Motor_SetSpeed(1,0);
31.     Motor_SetSpeed(2,0);
32. }

```

图 11 代码-驱动基本功能

蓝牙模块

串口通信里面，tx 发送信息，rx 接受信息，这里我使用了 pa9 口作为 tx，pa10 作为 rx
要注意，蓝牙的 TX 接 RX（pa10），RX 接 TX（pa9）

```

01. void Serial_Init(void)
02. {
03.     RCC_APB2PeriphClockCmd(RCC_APB2Periph_USART1, ENABLE);
04.     RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOA, ENABLE);
05.
06.
07.     GPIO_InitTypeDef GPIO_InitStructure;
08.     GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AF_PP;
09.     GPIO_InitStructure.GPIO_Pin = GPIO_Pin_9;//TX
10.     GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
11.     GPIO_Init(GPIOA, &GPIO_InitStructure);
12.
13.     GPIO_InitStructure.GPIO_Mode = GPIO_Mode_IPU;
14.     GPIO_InitStructure.GPIO_Pin = GPIO_Pin_10;//RX
15.     GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
16.     GPIO_Init(GPIOA, &GPIO_InitStructure);
17.
18.     USART_InitTypeDef USART_InitStructure;
19.     USART_InitStructure.USART_BaudRate = 9600;
20.     USART_InitStructure.USART_HardwareFlowControl = USART_HardwareFlowControl_None;
21.     USART_InitStructure.USART_Mode = USART_Mode_Tx | USART_Mode_Rx;
22.     USART_InitStructure.USART_Parity = USART_Parity_No;
23.     USART_InitStructure.USART_StopBits = USART_StopBits_1;
24.     USART_InitStructure.USART_WordLength = USART_WordLength_8b;
25.     USART_Init(USART1, &USART_InitStructure);
26.
27.     USART_ITConfig(USART1, USART_IT_RXNE, ENABLE);
28.
29.     NVIC_PriorityGroupConfig(NVIC_PriorityGroup_2);
30.
31.     NVIC_InitTypeDef NVIC_InitStructure;
32.     NVIC_InitStructure.NVIC_IRQChannel = USART1_IRQn;
33.     NVIC_InitStructure.NVIC_IRQChannelCmd = ENABLE;
34.     NVIC_InitStructure.NVIC_IRQChannelPreemptionPriority = 1;
35.     NVIC_InitStructure.NVIC_IRQChannelSubPriority = 1;
36.     NVIC_Init(&NVIC_InitStructure);
37.
38.     USART_Cmd(USART1, ENABLE);
39.
40. }

```

图 12 代码-蓝牙模块定义

然后通过发送指定命令遥控

自动避障

舵机定义：PBO 口，定时器 3

```

01. void PWMServo_Init(void)
02. {
03.
04.     RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOB, ENABLE);
05.     RCC_APB1PeriphClockCmd(RCC_APB1Periph_TIM3, ENABLE);
06.     GPIO_InitTypeDef GPIO_InitStructure;
07.     GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AF_PP;
08.     GPIO_InitStructure.GPIO_Pin = GPIO_Pin_0;
09.     GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
10.     GPIO_Init(GPIOB, &GPIO_InitStructure);
11.
12.     TIM_InternalClockConfig(TIM3);
13.
14.     TIM_TimeBaseInitTypeDef TIM_TimeBaseInitStructure;
15.     TIM_TimeBaseInitStructure.TIM_ClockDivision = TIM_CKD_DIV1;
16.     TIM_TimeBaseInitStructure.TIM_CounterMode = TIM_CounterMode_Up;
17.     TIM_TimeBaseInitStructure.TIM_Period = 20000 - 1;
18.     TIM_TimeBaseInitStructure.TIM_Prescaler = 72 - 1;
19.     TIM_TimeBaseInitStructure.TIM_RepetitionCounter = 0;
20.     TIM_TimeBaseInit(TIM3, &TIM_TimeBaseInitStructure);
21.
22.     TIM_OCInitTypeDef TIM_OCInitStructure;
23.     TIM_OCStructInit(&TIM_OCInitStructure);
24.     TIM_OCInitStructure.TIM_OCMode = TIM_OCMode_PWM1;
25.     TIM_OCInitStructure.TIM_OCPolarity = TIM_OCPolarity_High;
26.     TIM_OCInitStructure.TIM_OutputState = TIM_OutputState_Enable;
27.     TIM_OCInitStructure.TIM_Pulse = 0;
28.     TIM_OC3Init(TIM3, &TIM_OCInitStructure);
29.
30.     TIM_Cmd(TIM3, ENABLE);
31.
32. }
33.
34. void PWMServo_SetCompare3(uint16_t Compare)
35. {
36.     TIM_SetCompare3(TIM3, Compare);
37.
38. }

```

图 13 代码-舵机定义

舵机初始化封装和转弯角度

这里就直接使用江科大给出的计算公式就行

```

01. void Servo_Init(void)
02. {
03.     PWMServo_Init();
04. }
05.
06. void Servo_SetAngle(float Angle)
07. {
08.     PWMServo_SetCompare3(Angle / 180 * 2000 + 500);
09. }

```

图 14 代码-舵机角度计算

超声波模块的自动避障逻辑

```

01. Go_Ahead(); // 向前走
02. uint16_t front = Test_Distance();
03. Serial_SendNumber(front, 3);
04.
05. if(front < 15) {
06.     Car_Stop();
07.     Delay_ms(300);
08.
09.     uint8_t best_angle = 90;
10.     uint16_t max_dist = 0;
11.
12.     // 从左到右扫描 (45°~135°)
13.     for(uint8_t angle = 45; angle <= 135; angle += 15){
14.         Servo_SetAngle(angle);
15.         Delay_ms(300);
16.         uint16_t dist = Test_Distance();
17.         Serial_SendNumber(dist, 3);
18.         if(dist > max_dist){
19.             max_dist = dist;
20.             best_angle = angle;
21.         }
22.     }
23.
24.     if(max_dist > 15) {
25.         Servo_SetAngle(best_angle);
26.         Delay_ms(300);
27.
28.         if(best_angle < 85){
29.             Turn_Left(); // 如果舵机方向偏左, 小车也左转
30.             Delay_ms(600);
31.         } else if(best_angle > 95){
32.             Turn_Right(); // 偏右, 小车右转
33.             Delay_ms(600);
34.         }
35.
36.         Go_Ahead();
37.     } else {
38.         // 左右都不通, 执行后退 + 原地右转
39.         Go_Back();
40.         Delay_ms(800);
41.         Self_Right();
42.         Delay_ms(800);
43.         Go_Ahead();
44.     }
45. }
46.
47. Delay_ms(100);
48. break;
49.
50. }

```

图 15 代码-避障逻辑

自动循迹

使用红外检测 TCRT5000

使用的 PB5 到 PB8 口, 然后使用 GPIO_ReadInputDataBit 读取信号, 也就是 0 或 1, 0 代表没有检测到黑线, 1 代表检测到黑线

主流策略是检测到黑线, 红外光被吸收无返回, 车辆前进

红外检测要放在偏中间位置

但是为了显示不同，我们用检测不到红线前进，红外检测放在两边

只有黑线在车中间时前进，偏离再调整

缺点就是不如检测黑线前进稳定，容易出去继续前进

```
01. if(GPIO_ReadInputDataBit(GPIOB,GPIO_Pin_8)==0&&
02.     GPIO_ReadInputDataBit(GPIOB,GPIO_Pin_6)==0&&
03.     GPIO_ReadInputDataBit(GPIOB,GPIO_Pin_7)==0&&
04.     GPIO_ReadInputDataBit(GPIOB,GPIO_Pin_15)==0)
05. {
06.     Go_Ahead();
07. }else if(GPIO_ReadInputDataBit(GPIOB,GPIO_Pin_8)==1&&
08.     GPIO_ReadInputDataBit(GPIOB,GPIO_Pin_6)==1&&
09.     GPIO_ReadInputDataBit(GPIOB,GPIO_Pin_7)==1&&
10.     GPIO_ReadInputDataBit(GPIOB,GPIO_Pin_15)==1){
11.     Car_Stop();
12. }else if(GPIO_ReadInputDataBit(GPIOB,GPIO_Pin_8)==0&&
13.     GPIO_ReadInputDataBit(GPIOB,GPIO_Pin_6)==0&&
14.     GPIO_ReadInputDataBit(GPIOB,GPIO_Pin_7)==1&&
15.     GPIO_ReadInputDataBit(GPIOB,GPIO_Pin_15)==1){
16.     Self_Right();
17. }else if(GPIO_ReadInputDataBit(GPIOB,GPIO_Pin_8)==0&&
18.     GPIO_ReadInputDataBit(GPIOB,GPIO_Pin_6)==0&&
19.     GPIO_ReadInputDataBit(GPIOB,GPIO_Pin_7)==1&&
20.     GPIO_ReadInputDataBit(GPIOB,GPIO_Pin_15)==0){
21.     Turn_Right();
22. }else if(GPIO_ReadInputDataBit(GPIOB,GPIO_Pin_8)==0&&
23.     GPIO_ReadInputDataBit(GPIOB,GPIO_Pin_6)==0&&
24.     GPIO_ReadInputDataBit(GPIOB,GPIO_Pin_7)==0&&
25.     GPIO_ReadInputDataBit(GPIOB,GPIO_Pin_15)==1){
26.     Turn_Right();
27. }
28. else if(GPIO_ReadInputDataBit(GPIOB,GPIO_Pin_8)==0&&
29.     GPIO_ReadInputDataBit(GPIOB,GPIO_Pin_6)==1&&
30.     GPIO_ReadInputDataBit(GPIOB,GPIO_Pin_7)==0&&
31.     GPIO_ReadInputDataBit(GPIOB,GPIO_Pin_15)==0){
32.     Turn_Left();
33. }else if(GPIO_ReadInputDataBit(GPIOB,GPIO_Pin_8)==1&&
34.     GPIO_ReadInputDataBit(GPIOB,GPIO_Pin_6)==1&&
35.     GPIO_ReadInputDataBit(GPIOB,GPIO_Pin_7)==0&&
36.     GPIO_ReadInputDataBit(GPIOB,GPIO_Pin_15)==0){
37.     Self_Left();
38. }else if(GPIO_ReadInputDataBit(GPIOB,GPIO_Pin_8)==1&&
39.     GPIO_ReadInputDataBit(GPIOB,GPIO_Pin_6)==0&&
40.     GPIO_ReadInputDataBit(GPIOB,GPIO_Pin_7)==0&&
41.     GPIO_ReadInputDataBit(GPIOB,GPIO_Pin_15)==0){
42.     Turn_Left();
43. }
```

图 16 代码-循迹逻辑

最后将功能模块打包在主函数通过蓝牙连接发送对应命令调用

DATA1	功能
<u>0x30</u>	Car_Stop()
<u>0x31</u>	Go_Ahead()
<u>0x32</u>	Go_Back()
<u>0x33</u>	Turn_Left()
<u>0x34</u>	Turn_Right()
<u>0x35</u>	Self_Left()
<u>0x36</u>	Self_Right()
<u>0x37</u>	Servo_SetAngle(0)
<u>0x38</u>	Servo_SetAngle(90)
<u>0x39</u>	Servo_SetAngle(180)
<u>0x40</u>	红外循迹
<u>0x41</u>	自动避障
<u>0x42</u>	回到遥控模式

图 17 蓝牙操控功能表

第四章 系统调试与测试

1.1 硬件调试

电机驱动时要注意芯片接口的顺序，不然轮子会反向转动

由于蓝牙模块，舵机，电机驱动都需要 5V 电压，使用 4 节 5 号电池盒供电时，蓝牙连接上只能发送一次命令就会断开，发现是电池供电输入电压不够，用 MB03 电源模块使用充电宝供电就正常运行

舵机转向找完路线会偏移 45° ，需要复位操作

1.2 软件调试

蓝牙模块部分输入数据和输出用调试器的按键控制，所以需要再松开和按住发送一次数据

1.3 测试情况

可以正常通过蓝牙遥控小车行动，控制舵机角度

避障模块功能正常运行，能尽量不大偏移寻找出路

循迹模块由于是把黑线放入中心的策略，在黑线过短时容易超出去然后检测不到

可以通过调整 TCRT500 检测位置和更改策略解决

总结

本项目围绕 STM32F103C8T6 单片机为核心，设计并实现了一款具备蓝牙遥控、超声波避障与红外循迹功能的智能小车系统。在硬件方面，通过模块化设计，有效集成了驱动电路、感知模块与通信模块，实现了低成本、高性能集成的目标；在软件方面，使用 C 语言编程结合 Keil5 开发平台，完成了系统初始化、传感器数据处理、电机控制及逻辑判断等多个功能模块的程序编写。

整个项目过程中，不仅加深了对嵌入式系统开发流程的理解和掌握，还锻炼了电路搭建、模块调试、系统集成与问题解决的能力。系统成功实现了蓝牙遥控小车运动、自动避障及循迹导航等预期功能，运行稳定，响应及时，达到了设计目标。

